

Section 6 演算法實作：大規模鄰域搜索 (LNS) 優化框架

專注於「破壞與重建」機制與車輛數減量策略

1. 文件目的與適用範圍

本文件詳細說明如何利用大規模鄰域搜索 (LNS) 求解多校校車路徑問題 (Multi-School SBRP)。相較於遺傳演算法 (GA)，LNS 透過不斷「移除部分站點」並「重新插入」的過程，能更有效地在特定的局部解周邊進行深度搜索，特別適用於優化既有路徑的緊湊度與減少總派車數。

2. 演算法核心邏輯：破壞與重建

LNS 的核心在於透過 **Destroy Operator** (破壞算子) 移除當前解中的部分組成元素，再由 **Repair Operator** (重建算子) 將其修復，以此跳脫局部最佳解 (Local Optimum)。

3. 破壞算子設計 (Destroy Operators)

本實作採用兩種混合策略，以平衡「局部微調」與「全局減車」的需求：

1. 隨機站點移除 (Random Station Removal)：

邏輯：隨機選擇比例為 DESTROY_DEGREE (預設 50%) 的站點並從原路徑中抽離。目的：打亂現有路徑結構，釋放空間以利於重新排列組合。

2. 整路拆除 (Route Removal)：

邏輯：隨機選擇一條或多條完整路線，將該路線服務的所有站點全部移除。目的：強制將被拆除路線的乘客分配到其他既有路線，是實現「減車 (Bus Reduction)」目標的最強手段。

4. 重建算子設計 (Repair Operator)

採用貪婪插入啟發式 (Greedy Insertion Heuristic) 進行解的修復：

- 候選排序：將待插入站點集合進行隨機打亂，增加搜尋多樣性。
- 最優位置搜索：對於每個待插入站點，遍歷所有現有路徑的每一個可能插入點 (Insertion Point)。
- 多目標評估：

Option 1 (現有路徑)：計算插入後的行駛時間、載重與公平性懲罰，選擇增加總成本最小的位置。**Option 2** (新增路徑)：若現有路徑皆無法容納或成本過高，且總車數未達 MAX_TOTAL_BUSES，則嘗試為其建立一條新路徑。

5. 接受準則與優化策略

接受準則 (Acceptance Criterion)：本實作採用 Hill Climbing (爬山法)，僅在重建後的解其 solution_cost 優於當前最佳解時才予以接受。路線合併優化 (Route Merging)：在每次重建完成後，呼叫 try_merge_routes 函數。該函數會嘗試將兩條人數較少的路線合併，這在 LNS 迭代中能顯著提升車輛載重率。

6. 演算法流程 (Algorithm Pipeline)

Algorithm: LNS for SBRP

Input: Schools, Stations, Iterations (300), Destroy Degree (0.5)

Output: Best Solution Found

1. Initial Solution: Generate using Greedy Nearest Neighbor.
2. For iteration $it = 1$ to 300:
 - a. Current Solution Backup.
 - b. If $\text{rand}() < 0.3$:
 - Destroy: Remove 1 entire route.
 - Else:
 - Destroy: Randomly remove 50% of served stations.
 - c. Repair: Re-insert removed stations using Greedy Insertion.
 - d. Post-process: Attempt to merge small routes (try_merge_routes).
 - e. If temp_sol.feasible AND $\text{cost}(\text{temp_sol}) < \text{cost}(\text{best_sol})$:
 - Update $\text{best_sol} = \text{temp_sol}$.
3. Return Best Solution.

7. 關鍵參數說明與調校建議

DESTROY_DEGREE：設定為 0.5。若設定太低，搜尋範圍過窄，易陷入局部解；若設定太高，則演算法退化為隨機搜尋，難以收斂。**MAX_TOTAL_BUSES**：硬性約束上限（預設 6 台）。LNS 透過「整路拆除」算子與「路線合併」策略，會主動嘗試以少於此上限的車數完成任務。

try_merge_routes：這是維持高品質解的關鍵，建議在每次重建後都執行，以確保系統能自動發現「兩條短路徑合併為一條長路徑」的機會。

8. 實作建議與注意事項

1. 初始解品質很重要：LNS 是從既有解周邊搜尋，若初始解過差，後續破壞與重建會花更多時間修補可行性。
2. **Destroy** 與 **Repair** 要保持平衡：破壞太小會搜尋不足，破壞太大会讓重建成本過高。
3. 適合作為其他方法的初始化器：LNS 產生的可行路線可供 Matheuristic 或 Dantzig-Wolfe 作為初始欄位，降低 master problem infeasible 的風險。